

# OCA I2C Controller — Functional Specification

---

## SMC I2C Controller — Functional Specification

### Contents

1. Purpose and scope
2. Conformance and references
3. Feature summary
4. Configuration parameters
  - 4.1 CCI parameters (primary interface)
  - 4.2 `i2c_controller_cfg` struct (defaults + fixed constants)
5. Block diagram, port list, and flow diagrams
  - 5.1 Block diagram (ASCII)
  - 5.2 Ports
  - 5.3 Flow diagrams
6. Theory of operation
  - 6.1 Operating principle
  - 6.2 Architectural concepts
  - 6.3 Controller-Mode datapath
  - 6.4 Target-Mode datapath
  - 6.5 Loosely-timed timing
7. Register map
  - 7.1 CTRL (0x10)
  - 7.2 FDATA format entry (0x1C, WO)
  - 7.3 FIFO\_CTRL (0x20, WO, self-clearing)
  - 7.4 STATUS (0x14, RO, computed)
  - 7.5 ACQDATA (0x58, RO) — `SIGNAL[10:8]` codes
8. Functional behaviour
  - 8.1 Controller write sequence
  - 8.2 Controller read sequence
  - 8.3 Repeated START
  - 8.4 Target write / read (back door)
  - 8.5 FIFO thresholds and levels
9. Interrupts
10. Reset behaviour
11. Bus interface (TLM-2.0 / AXI4-Lite)
  - 11.1 Transport function
  - 11.2 Error responses
  - 11.3 Debug transport
  - 11.4 DMI
  - 11.5 AXI sideband
12. Programming model
  - 12.1 Controller write (poll)
  - 12.2 Controller read (poll)

12.3 Target setup

13. Compliance matrix

14. Revision history

15. Glossary

# SMC I2C Controller — Functional Specification

---

**Document:** `01_I2C_CONTROLLER_Specification.md` **Module:** `smc::i2c_controller` (SystemC/TLM-2.0 Loosely-Timed model) **Spec base:** OpenTitan-derived OCA `i2c` core (Controller / Target / Hybrid / Monitor) **RTL reference:** `hw/comp/i2c/rtl/` (register map `hw/comp/i2c/data/registers/rd1/i2c.rdl`) **Status:** Design phase — document governs the `peripherals/i2c_controller/` model **Companion docs:** - `02_I2C_CONTROLLER_LowLevel_Design.md` — internal SystemC implementation - `03_I2C_CONTROLLER_Test_Plan.md` — verification strategy and test list

---

## Contents

---

1. Purpose and scope
  2. Conformance and references
  3. Feature summary
  4. Configuration parameters
  5. Block diagram, port list, and flow diagrams
  6. Theory of operation
  7. Register map
  8. Functional behaviour
  9. Interrupts
  10. Reset behaviour
  11. Bus interface (TLM-2.0 / AXI4-Lite)
  12. Programming model
  13. Compliance matrix
  14. Revision history
  15. Glossary
- 

## 1. Purpose and scope

---

The SMC I2C Controller is an OpenTitan-derived **I2C controller/target** peripheral for the SMC chiplet. The `i2c_wrap` platform wrapper packs `NUM_I2CS` (default 3) identical `i2c` cores plus a small wrapper-enable register behind one AXI slave; each core occupies a 0x200-byte register window (`i2c[NUM_I2CS] @0x0 += 0x200, i2c_ctrl @0xe00`).

This specification defines the **externally-observable behaviour of one `i2c` core** as implemented by the SystemC/TLM-2.0 loosely-timed functional model: configuration parameters, register map, the Controller-Mode and Target-Mode datapaths, the four FIFOs, the 20-source interrupt block, reset, and the TLM-2.0 bus contract. It is the contract that:

- Firmware (register-mapped driver code) programs against,
- The RTL (`hw/comp/i2c`) implements at gate level, and
- The SystemC model (`peripherals/i2c_controller/`) implements at transaction level.

The model is a **functional** model: register semantics and byte-level data flow are reproduced faithfully and are bit-compatible with the RDL register layout, while bit-level SCL/SDA signalling, clock stretching, arbitration timing, and the analog timing/timeout parameters are firmware-visible storage with no modelled waveform (see §13). Modelling one core (rather than the `i2c_wrap` bundle) mirrors how `smc::uart` models a single UART instance; the platform integrator instantiates as many as the wrapper requires.

Internal implementation choices (process topology, FIFO modelling, the FMT-drain engine, quantum-keeper usage) are documented in [02\\_I2C\\_CONTROLLER\\_LowLevel1\\_Design.md](#).

## 2. Conformance and references

| Source                                                          | Authority                                     |
|-----------------------------------------------------------------|-----------------------------------------------|
| <code>hw/comp/i2c/data/registers/rd1/i2c.rdl</code>             | <b>Ground-truth</b> register map (this model) |
| <code>hw/comp/i2c/doc/architecture.adoc</code>                  | Block diagram, FSMs, FIFOs                    |
| <code>hw/comp/i2c/doc/programming.adoc</code>                   | Controller/Target programming sequences       |
| <code>hw/comp/i2c/doc/memmap.adoc</code>                        | Register offsets                              |
| <code>hw/periph/i2c_wrap/data/registers/rd1/i2c_wrap.rdl</code> | Multi-instance packing                        |
| <code>hw/periph/i2c_wrap/data/registers/rd1/i2c_ctrl.rdl</code> | Wrapper enable register (out of scope here)   |
| I2C-bus specification (NXP UM10204) / SMBus 3.x                 | Industry-standard protocol baseline           |

The model is **bit-compatible** with the RDL register layout and protocol-compatible with the OpenTitan I2C programming model. All deliberate deviations (abstracted timing) are listed in §13.

### 3. Feature summary

| Feature                  | Value / behaviour                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------------------------|
| Roles                    | Controller, Target, Hybrid (both), Monitor — via <code>CTRL.ENABLEHOST</code> / <code>CTRL.ENABLETARGET</code>  |
| Controller TX (FMT) FIFO | Configurable depth (default 64 entries); <code>fmt_fifo_depth</code> CCI param                                  |
| Controller RX FIFO       | Configurable depth (default 64 bytes); <code>rx_fifo_depth</code> CCI param                                     |
| Target TX FIFO           | Configurable depth (default 64 bytes); <code>tx_fifo_depth</code> CCI param                                     |
| Target RX (ACQ) FIFO     | Configurable depth (default 64 entries); <code>acq_fifo_depth</code> CCI param                                  |
| Addressing               | 7-bit; two programmable target addresses + masks ( <code>TARGET_ID</code> )                                     |
| Format flags             | START / STOP / READB / RCONT / NAKOK per FMT entry ( <code>FDATA</code> )                                       |
| ACQ signalling           | Start / Stop / Restart / Data / NackData / NackStart / Error ( <code>ACQDATA.SIGNAL</code> )                    |
| SMBus features           | SMBus enable / suspend / alert ( <code>SMBUS_CTRL</code> , <code>SMBUS_STATUS</code> ) — abstracted             |
| Interrupt sources        | 20 (mix of hardware-computed level bits and latched write-1-to-clear events)                                    |
| Timing / timeouts        | <code>TIMING0..4</code> , <code>TIMEOUT_CTRL</code> , host/target/NAK-handler timeouts — storage only           |
| Bus interface            | TLM-2.0 LT target socket (AXI4-Lite, 32-bit data, 32-bit address)                                               |
| Register window          | 512 bytes ( <code>0x000</code> .. <code>0x1FF</code> ); registers occupy <code>0x00</code> .. <code>0x80</code> |
| Reset polarity           | Active-low asynchronous ( <code>rst_n_i</code> )                                                                |

### 4. Configuration parameters

#### 4.1 CCI parameters (primary interface)

The model exposes the following OSCI CCI ( `cci_configuration` ) parameters. These are the **primary** configuration interface; test benches and platform integrators use the CCI broker to set them.

| CCI parameter                | Type                                                                           | Mutability | Default | Notes                                                                                                                     |
|------------------------------|--------------------------------------------------------------------------------|------------|---------|---------------------------------------------------------------------------------------------------------------------------|
| <code>fmt_fifo_depth</code>  | <code>cci_param&lt;unsigned&gt;</code><br>( <code>CCI_IMMUTABLE_PARAM</code> ) | Immutable  | 64      | Controller TX (FMT) FIFO depth in entries. Maps to RTL <code>FMTFIFO_DEPTH</code> . Must be $\geq 1$ .                    |
| <code>rx_fifo_depth</code>   | <code>cci_param&lt;unsigned&gt;</code><br>( <code>CCI_IMMUTABLE_PARAM</code> ) | Immutable  | 64      | Controller RX FIFO depth in bytes. Maps to RTL <code>RXFIFO_DEPTH</code> . Must be $\geq 1$ .                             |
| <code>tx_fifo_depth</code>   | <code>cci_param&lt;unsigned&gt;</code><br>( <code>CCI_IMMUTABLE_PARAM</code> ) | Immutable  | 64      | Target TX FIFO depth in bytes. Maps to RTL <code>TXFIFO_DEPTH</code> . Must be $\geq 1$ .                                 |
| <code>acq_fifo_depth</code>  | <code>cci_param&lt;unsigned&gt;</code><br>( <code>CCI_IMMUTABLE_PARAM</code> ) | Immutable  | 64      | Target RX (ACQ) FIFO depth in entries. Maps to RTL <code>ACQFIFO_DEPTH</code> . Must be $\geq 1$ .                        |
| <code>access_delay_ns</code> | <code>cci_param&lt;double&gt;</code>                                           | Mutable    | 2.0     | Delay added to <code>sc_time&amp;</code> <code>delay</code> in <code>b_transport</code> . Approximates AXI4-Lite latency. |
| <code>xfer_delay_ns</code>   | <code>cci_param&lt;double&gt;</code>                                           | Mutable    | 100.0   | Modelled latency between an FMT push (or host-enable) and the transaction executing.                                      |

Hierarchical param names follow the standard CCI convention `<broker_prefix>.<sc_module_name>.<param_name>`, e.g.:

```
broker.set_preset_cci_value("top.i2c0.rx_fifo_depth", cci::cci_value(128u));
auto h = broker.get_param_handle("top.i2c0.xfer_delay_ns");
h.set_cci_value(cci::cci_value(20.0)); // mutable
```

Each FIFO-depth of 0 is rejected at construction with `SC_REPORT_FATAL`.

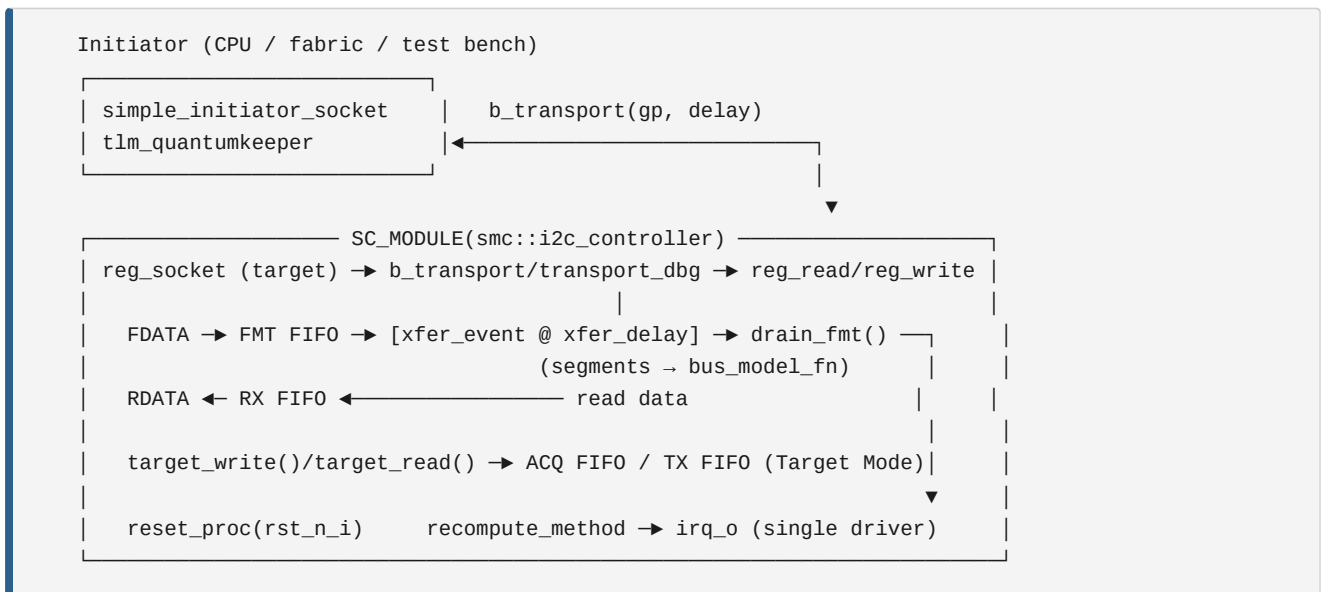
## 4.2 `i2c_controller_cfg` struct (defaults + fixed constants)

The constructor accepts an optional `i2c_controller_cfg cfg` whose fields supply the **default values** for the CCI params (a broker preset set before construction wins). The struct also holds the fixed address-map constants (never CCI params because they must not vary at run-time):

| Constant    | Value | Description                                                               |
|-------------|-------|---------------------------------------------------------------------------|
| WINDOW_SIZE | 0x200 | 512-byte decoded window (per-core stride from <code>i2c_wrap.rdl</code> ) |
| REG_WIDTH   | 4     | Register stride (bytes)                                                   |

## 5. Block diagram, port list, and flow diagrams

### 5.1 Block diagram (ASCII)



The SCL/SDA lines are not simulated at bit level. In Controller Mode the remote target(s) are emulated by a test-bench `bus_model_fn`; in Target Mode an external controller is emulated by the `target_write` / `target_read` back doors.

## 5.2 Ports

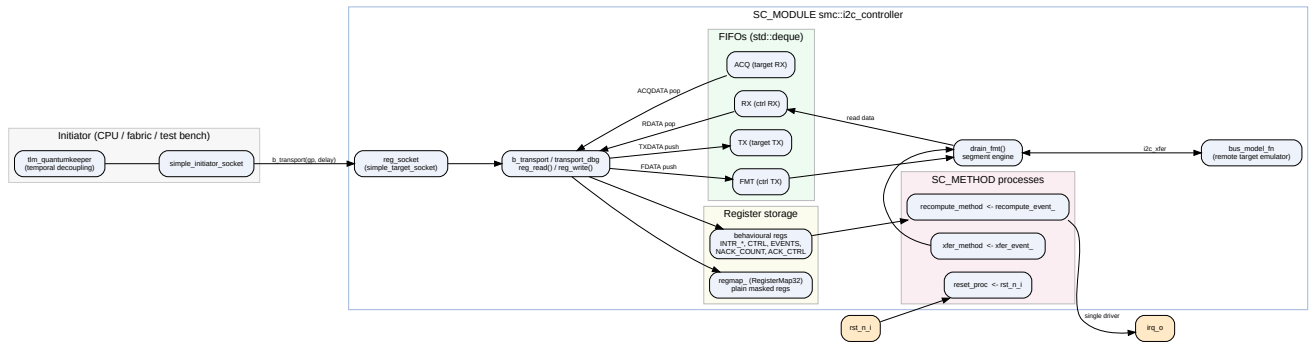
| Port       | Direction | Type                                            | Description                                                   |
|------------|-----------|-------------------------------------------------|---------------------------------------------------------------|
| reg_socket | target    | tlm_utils::simple_target_socket<i2c_controller> | TLM-2.0 LT register-access socket (AXI4-Lite-style)           |
| rst_n_i    | input     | sc_in<bool>                                     | Active-low asynchronous reset                                 |
| irq_o      | output    | sc_out<bool>                                    | Interrupt output (active-high; OR of enabled INTR_STATE bits) |

The bit-level SDA/SCL pads of the RTL core are intentionally **not** exposed; their function is abstracted into the bus-model / back-door API (see §6 and 02\_\* §12).

## 5.3 Flow diagrams

The following diagrams show the structural architecture and the main dynamic flows. They complement the ASCII block diagram in §5.1 and the datapath pseudocode in §6.3–§6.4. Each figure is a rendered **SVG** (`doc/figures/`) so it displays in any Markdown viewer, including Cursor's built-in preview; the SVGs are generated from the Graphviz sources next to them (`dot -Tsvg NN_name.dot -o NN_name.svg`). The equivalent Mermaid source — which some viewers (GitHub/GitLab/VS Code) render inline — is kept as a collapsible text fallback under each figure.

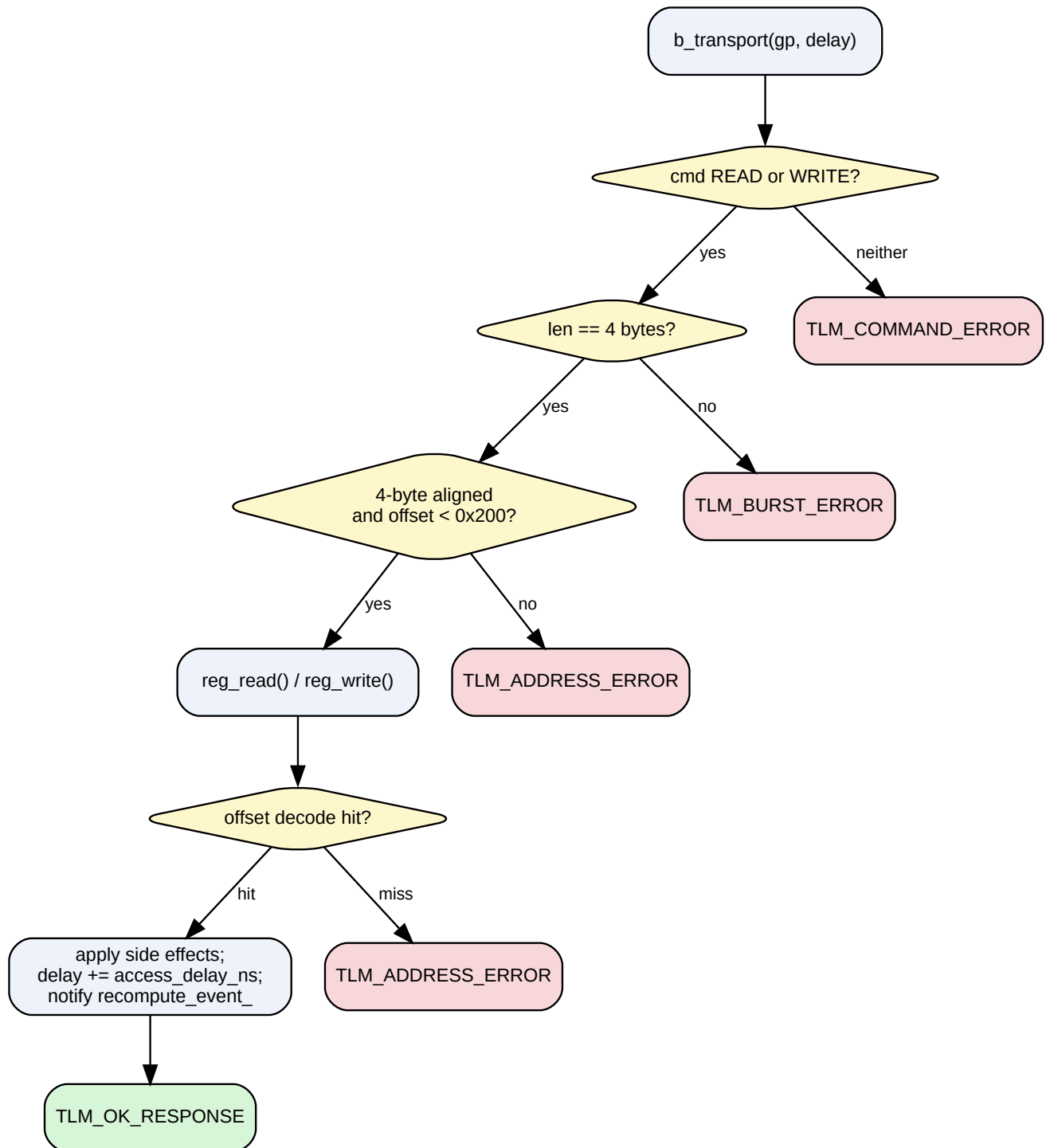
### 5.3.1 Structural architecture



Structural architecture of `smc::i2c_controller`: an initiator (CPU/fabric/test bench) drives `reg_socket` via `b_transport` through a `tlm_quantumkeeper`; inside the module `b_transport/transport_dbg` decode into `regmap_` and the behavioural registers, push/pop the `FMT/RX/TX/ACQ` FIFOs, `drain_fmt` exchanges `i2c_xfer` segments with the `bus_model`, and `reset_proc/xfer_method/recompute_method` drive `irq_o` under single-driver discipline.

► Text fallback (Mermaid source)

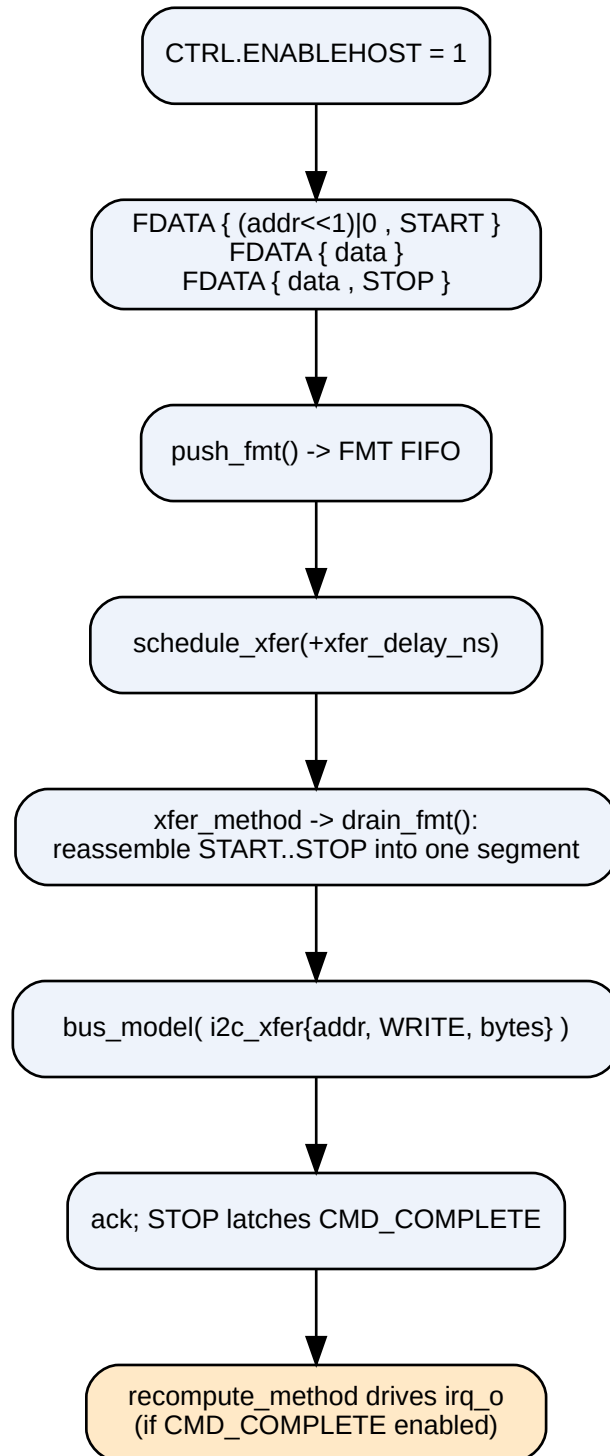
### 5.3.2 Register-access flow (every transaction, §11)



Register-access flow: `b_transport` checks the command is READ or WRITE (else `TLM_COMMAND_ERROR`), the length is 4 bytes (else `TLM_BURST_ERROR`), and the offset is 4-byte aligned and below 0x200 (else `TLM_ADDRESS_ERROR`); `reg_read/reg_write` then decode the offset, a miss returns `TLM_ADDRESS_ERROR` while a hit applies side effects, adds `access_delay_ns`, and returns `TLM_OK_RESPONSE`.

► Text fallback (Mermaid source)

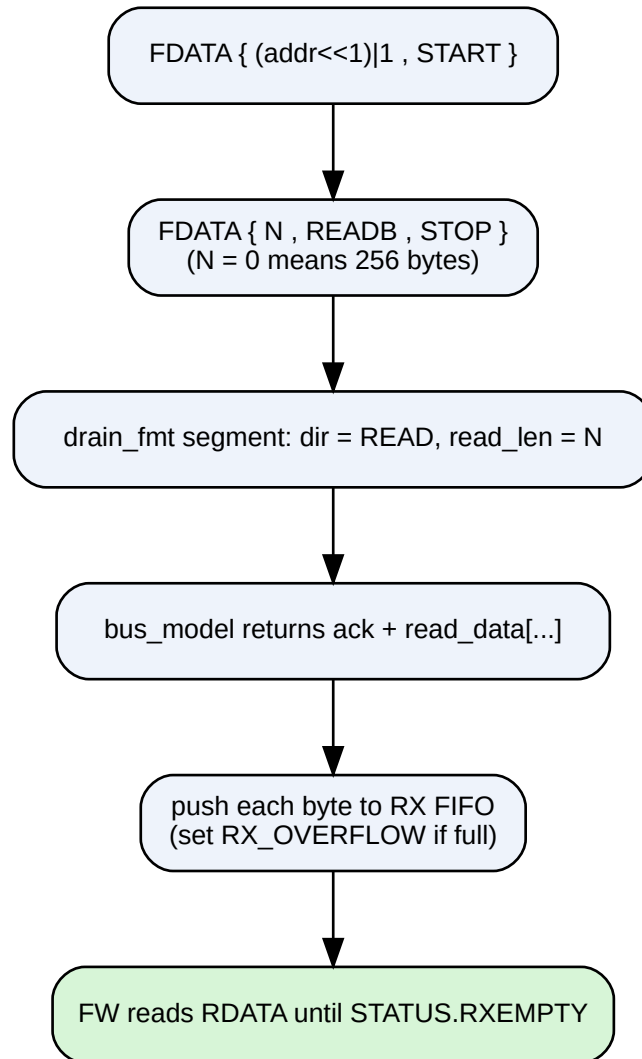
### 5.3.3 Controller-Mode WRITE (§6.3, §8.1)



Controller-Mode WRITE flow: firmware sets `CTRL.ENABLEHOST`, writes FDATA entries (addr with START, data, data with STOP) that push\_fmt into the FMT FIFO and schedule an xfer; on xfer\_event\_drain\_fmt reassembles START..STOP into one segment and calls bus\_model with an i2c\_xfer write, the STOP latches CMD\_COMPLETE, and recompute\_method drives irq\_o high when CMD\_COMPLETE is enabled.

► Text fallback (Mermaid source)

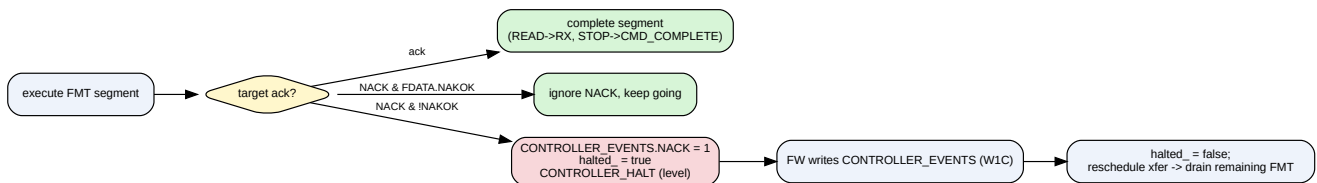
### 5.3.4 Controller-Mode READ (§6.3, §8.2)



Controller-Mode READ flow: firmware writes an FDATA START entry with the read address then an FDATA entry carrying byte count N with READB and STOP (N=0 means 256); drain\_fmt issues an i2c\_xfer read of read\_len N, the bus\_model returns ack plus read\_data, each returned byte is pushed to the RX FIFO (setting RX\_OVERFLOW if full), and firmware reads RDATA until STATUS.RXEMPTY.

► Text fallback (Mermaid source)

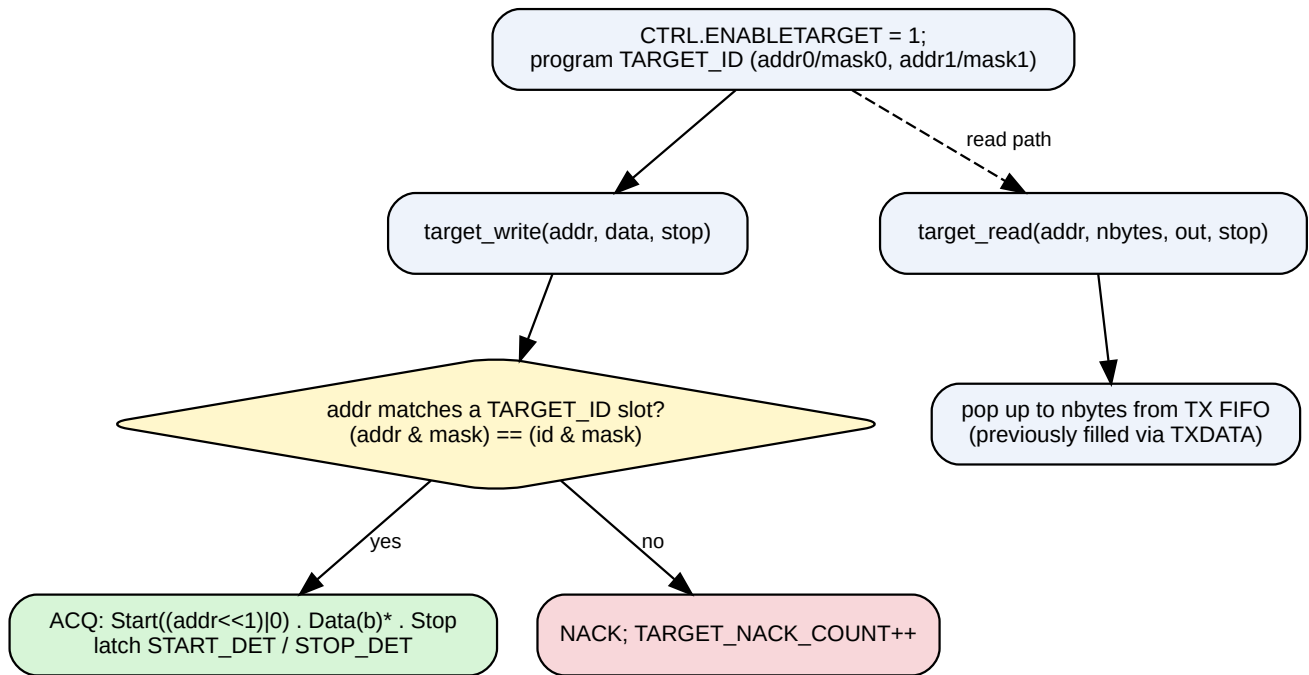
### 5.3.5 NACK -> halt -> resume (§6.2, §8.1)



NACK/halt/resume flow: after executing a segment, an ack completes it (read data to RX, STOP latches CMD\_COMPLETE); a NACK with FDATA.NAKOK is ignored and the stream continues; a NACK without NAKOK sets CONTROLLER\_EVENTS.NACK, sets halted\_ and the level-sensitive CONTROLLER\_HALT, and firmware clears CONTROLLER\_EVENTS (W1C) to drop halted\_ and reschedule the xfer to drain the remaining FMT entries.

► Text fallback (Mermaid source)

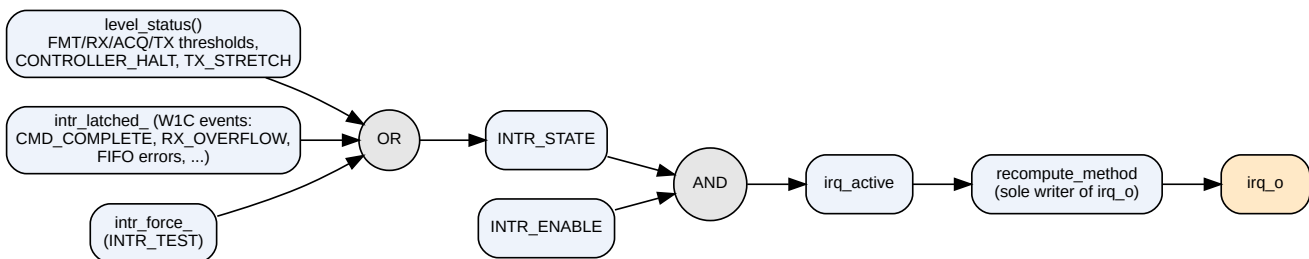
### 5.3.6 Target Mode (§6.4, §8.4)



*Target-Mode flow: with CTRL.ENABLETARGET set and TARGET\_ID programmed, an external controller (back door) issues target\_write; a matching address fills the ACQ FIFO with Start, Data, and Stop entries and latches START/STOP\_DETECT, while a non-match returns NACK and increments TARGET\_NACK\_COUNT; a target\_read drains up to nbytes from the TX FIFO previously filled via TXDATA.*

► Text fallback (Mermaid source)

### 5.3.7 Interrupt aggregation (§9)



*Interrupt aggregation: level\_status() (FMT/RX/ACQ/TX thresholds, CONTROLLER\_HALT, TX\_STRETCH), the W1C intr\_latched\_ events (CMD\_COMPLETE, RX\_OVERFLOW, FIFO errors, ...), and intr\_force\_ from INTR\_TEST are ORed into INTR\_STATE, ANDed with INTR\_ENABLE to form irq\_active, which recompute\_method (the sole writer) drives onto irq\_o.*

► Text fallback (Mermaid source)

**Mental model:** the register path (`b_transport`) is decoupled and effectively instantaneous — it only touches registers/FIFOs and annotates `access_delay_ns`. The protocol work (the actual I2C transaction) happens *later*, on `xfer_event_`, in `drain_fmt`, against the abstracted bus. That annotate-now / execute-on-event split is what makes this a loosely-timed model and is why the quantum keeper lives in the initiator, not the peripheral.

## 6. Theory of operation

### 6.1 Operating principle

The I2C core moves bytes between firmware-visible FIFOs and the (abstracted) I2C bus in one of two roles, selected by `CTRL`:

- **Controller Mode** (`CTRL.ENABLEHOST = 1`): firmware pushes *format entries* into the Controller TX FIFO (“FMT”) through `FDATA`. Each entry is a byte plus START/STOP/READB/ RCONT/NAKOK flags. The engine reassembles the entries into per-address **segments** and presents each to the bus; read data is pushed into the Controller RX FIFO (`RDATA`).
- **Target Mode** (`CTRL.ENABLETARGET = 1`): an external controller addresses this device. A matching address (per `TARGET_ID`) fills the Target RX FIFO (“ACQ”, read via `ACQDATA`) with signalled entries; read requests drain the Target TX FIFO (filled via `TXDATA`).

### 6.2 Architectural concepts

| Concept                | Definition                                                                                                                                                                            |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FMT FIFO</b>        | Controller TX format queue. Each entry = {byte, start, stop, readb, rcont, nakok} written through <code>FDATA</code> .                                                                |
| <b>RX FIFO</b>         | Controller receive queue (bytes). Popped by reading <code>RDATA</code> .                                                                                                              |
| <b>TX FIFO</b>         | Target transmit queue (bytes). Filled by writing <code>TXDATA</code> ; drained on a Target-Mode read.                                                                                 |
| <b>ACQ FIFO</b>        | Target receive/acquisition queue. Each entry = {byte, signal} read through <code>ACQDATA</code> .                                                                                     |
| <b>Segment</b>         | One addressed phase: START/repeated-START + address byte, then the write bytes or requested read length, up to the next START or STOP.                                                |
| <b>Bus model</b>       | Test-bench callback ( <code>i2c_bus_model_fn</code> ) that, given a segment ( <code>i2c_xfer</code> ), sets the ACK and, for reads, supplies the read bytes.                          |
| <b>Controller halt</b> | An address/data NACK (without NAKOK) sets a <code>CONTROLLER_EVENTS</code> bit, halts the FMT engine, and raises <code>CONTROLLER_HALT</code> . Clearing the events (W1C) resumes it. |

### 6.3 Controller-Mode datapath

```

FDATA write → FMT FIFO → [xfer_event @ xfer_delay_ns] → drain_fmt()
  reassemble entries into segments:
    START(addr|R/W) → open segment
    data byte       → append to write payload (write) or add to read length (read)
    STOP            → execute segment, raise CMD_COMPLETE
  execute(segment): bus_model_fn(xfer)
    ack == false && !nakok → CONTROLLER_EVENTS.NACK, halt
    dir == Read           → push returned bytes into RX FIFO (RX_OVERFLOW if full)

```

## 6.4 Target-Mode datapath

```
external write: target_write(addr, data, stop)
    addr matches TARGET_ID?  no → NACK, TARGET_NACK_COUNT++
                             yes → ACQ: Start(addr<<1) · Data(b)* · [Stop]
                                TARGET_EVENTS.START_DETECT / STOP_DETECT

external read:  target_read(addr, nbytes, out, stop)
    addr matches?      no → NACK, TARGET_NACK_COUNT++
                       yes → ACQ: Start(addr<<1|1); drain up to nbytes from TX FIFO
```

## 6.5 Loosely-timed timing

`b_transport` never calls `wait()`; it only adds `access_delay_ns` to the annotated `delay` (temporal decoupling is the initiator's responsibility — the test-bench driver owns the `t1m_quantumkeeper`). The modelled transaction latency `xfer_delay_ns` is realised by scheduling an internal `sc_event`, so a Controller-Mode transfer completes a bounded time after the FMT push rather than instantaneously.

## 7. Register map

All registers are 32-bit and word-aligned within the 512-byte window. Offsets `0x00..0x80` are decoded; any other aligned offset inside the window returns `TLM_ADDRESS_ERROR_RESPONSE` (same policy as the UART model).

| Offset | Register            | Access | Description                                                        |
|--------|---------------------|--------|--------------------------------------------------------------------|
| 0x00   | INTR_STATE          | RW     | Interrupt state (level bits read-only; latched event bits are W1C) |
| 0x04   | INTR_ENABLE         | RW     | Per-source interrupt enable                                        |
| 0x08   | INTR_TEST           | WO     | Force interrupts (level bits held while set; event bits latched)   |
| 0x0C   | SMBUS_CTRL          | RW     | SMBus suspend / alert drive (abstracted)                           |
| 0x10   | CTRL                | RW     | Mode/enable select (see §7.1)                                      |
| 0x14   | STATUS              | RO     | FIFO full/empty + idle flags (computed)                            |
| 0x18   | RDATA               | RO     | Pop Controller RX FIFO (returns 0 when empty)                      |
| 0x1C   | FDATA               | WO     | Push a Controller TX (FMT) format entry (see §7.2)                 |
| 0x20   | FIFO_CTRL           | WO     | Self-clearing FIFO resets (see §7.3)                               |
| 0x24   | HOST_FIFO_CONFIG    | RW     | RX_THRESH[11:0] / FMT_THRESH[27:16]                                |
| 0x28   | TARGET_FIFO_CONFIG  | RW     | TX_THRESH[11:0] / ACQ_THRESH[27:16]                                |
| 0x2C   | HOST_FIFO_STATUS    | RO     | FMTLVL[11:0] / RXLVL[27:16] (computed)                             |
| 0x30   | TARGET_FIFO_STATUS  | RO     | TXLVL[11:0] / ACQLVL[27:16] (computed)                             |
| 0x34   | OVRD                | RW     | SDA/SCL override (abstracted storage)                              |
| 0x38   | VAL                 | RO     | Oversampled SCL/SDA (abstracted → 0)                               |
| 0x3C   | TIMING0             | RW     | THIGH / TLOW (storage)                                             |
| 0x40   | TIMING1             | RW     | T_R / T_F (storage)                                                |
| 0x44   | TIMING2             | RW     | TSU_STA / THD_STA (storage)                                        |
| 0x48   | TIMING3             | RW     | TSU_DAT / THD_DAT (storage)                                        |
| 0x4C   | TIMING4             | RW     | TSU_STO / T_BUF (storage)                                          |
| 0x50   | TIMEOUT_CTRL        | RW     | VAL / MODE / EN (storage)                                          |
| 0x54   | TARGET_ID           | RW     | ADDRESS0/MASK0/ADDRESS1/MASK1 (7-bit each)                         |
| 0x58   | ACQDATA             | RO     | Pop Target RX (ACQ) FIFO: ABYTE[7:0] + SIGNAL[10:8]                |
| 0x5C   | TXDATA              | WO     | Push Target TX FIFO                                                |
| 0x60   | HOST_TIMEOUT_CTRL   | RW     | VAL (storage)                                                      |
| 0x64   | TARGET_TIMEOUT_CTRL | RW     | VAL / EN (storage)                                                 |
| 0x68   | TARGET_NACK_COUNT   | RW     | Saturating 8-bit NACK counter; <b>read-clear</b>                   |
| 0x6C   | TARGET_ACK_CTRL     | RW     | NBYTES[8:0] ; writing bit 31 pulses a target NACK                  |

| Offset | Register                  | Access | Description                                                             |
|--------|---------------------------|--------|-------------------------------------------------------------------------|
| 0x70   | ACQ_FIFO_NEXT_DATA        | RO     | Next ACQ byte without popping (computed)                                |
| 0x74   | HOST_NACK_HANDLER_TIMEOUT | RW     | VAL / EN (storage)                                                      |
| 0x78   | CONTROLLER_EVENTS         | RW     | NACK / ... / ARBITRATION_LOST (W1C; gate <code>CONTROLLER_HALT</code> ) |
| 0x7C   | TARGET_EVENTS             | RW     | TX_PENDING / ... / START_DETECT / STOP_DETECT (W1C)                     |
| 0x80   | SMBUS_STATUS              | RO     | SMBus suspend / alert status (abstracted → 0)                           |

## 7.1 CTRL (0x10)

| Bit | Field                       | Description                                         |
|-----|-----------------------------|-----------------------------------------------------|
| 0   | ENABLEHOST                  | Enable Controller Mode (drains the FMT FIFO)        |
| 1   | ENABLETARGET                | Enable Target Mode (accepts back-door transactions) |
| 2   | LLPBK                       | Line-loopback (abstracted storage)                  |
| 3   | NACK_ADDR_AFTER_TIMEOUT     | (storage)                                           |
| 4   | ACK_CTRL_EN                 | Enable software ACK control (storage)               |
| 5   | MULTI_CONTROLLER_MONITOR_EN | (storage)                                           |
| 6   | TX_STRETCH_CTRL_EN          | (storage)                                           |
| 7   | ACQ_START_STOP_EN           | (storage)                                           |

## 7.2 FDATA format entry (0x1C, WO)

| Bit | Field | Description                                                                                                  |
|-----|-------|--------------------------------------------------------------------------------------------------------------|
| 7:0 | FBYTE | Data byte, or (for the address phase) {addr[6:0], R/W}, or read length when <code>READB</code> set (0 ⇒ 256) |
| 8   | START | Issue START/repeated-START before this byte (byte is the address)                                            |
| 9   | STOP  | Issue STOP after this byte (closes the segment; raises <code>CMD_COMPLETE</code> )                           |
| 10  | READB | This entry requests a read of <code>FBYTE</code> bytes                                                       |
| 11  | RCONT | Read continues (no NACK on last byte) — accepted; not separately modelled                                    |
| 12  | NAKOK | A NACK on this segment is expected and does not halt the controller                                          |

### 7.3 FIFO\_CTRL (0x20, WO, self-clearing)

| Bit | Field  | Effect                             |
|-----|--------|------------------------------------|
| 0   | RXRST  | Clear the Controller RX FIFO       |
| 1   | FMTRST | Clear the Controller TX (FMT) FIFO |
| 7   | ACQRST | Clear the Target RX (ACQ) FIFO     |
| 8   | TXRST  | Clear the Target TX FIFO           |

### 7.4 STATUS (0x14, RO, computed)

| Bit | Field            | Meaning                                              |
|-----|------------------|------------------------------------------------------|
| 0   | FMTFULL          | FMT FIFO full                                        |
| 1   | RXFULL           | RX FIFO full                                         |
| 2   | FMTEMTY          | FMT FIFO empty                                       |
| 3   | HOSTIDLE         | FMT empty and controller not halted                  |
| 4   | TARGETIDLE       | Target idle (functionally always 1 in this LT model) |
| 5   | RXEMPTY          | RX FIFO empty                                        |
| 6   | TXFULL           | TX FIFO full                                         |
| 7   | ACQFULL          | ACQ FIFO full                                        |
| 8   | TXEMPTY          | TX FIFO empty                                        |
| 9   | ACQEMPTY         | ACQ FIFO empty                                       |
| 10  | ACK_CTRL_STRETCH | Software ACK-control stretch (not modelled → 0)      |

### 7.5 ACQDATA (0x58, RO) — SIGNAL[10:8] codes

| Code | Signal    | Meaning                                       |
|------|-----------|-----------------------------------------------|
| 0x0  | Data      | Ordinary ACKed data byte                      |
| 0x1  | Start     | Address byte preceded by a START              |
| 0x2  | Stop      | STOP after ACKed data                         |
| 0x3  | Restart   | Address byte preceded by a repeated START     |
| 0x4  | NackData  | NACKed data byte                              |
| 0x5  | NackStart | Address byte whose following data were NACKed |
| 0x6  | Error     | Abnormal transaction termination              |

## 8. Functional behaviour

### 8.1 Controller write sequence

1. CTRL.ENABLEHOST = 1.
2. Write FDATA = {addr<<1 | 0, START} (address, write).

3. Write one `FDATA` per data byte; set `STOP` on the last.
4. `xfer_delay_ns` later the engine executes the segment against the bus model. On ACK the write completes; `STOP` raises `CMD_COMPLETE`.
5. An address/data NACK without `NAKOK` sets `CONTROLLER_EVENTS.NACK`, raises `CONTROLLER_HALT`, and halts further draining until the event is cleared (W1C).

## 8.2 Controller read sequence

1. `CTRL.ENABLEHOST = 1`.
2. Write `FDATA = {addr<<1 | 1, START}` (address, read).
3. Write `FDATA = {N, READB, STOP}` to request `N` bytes ( $N = 0 \Rightarrow 256$ ).
4. On ACK the bus model supplies up to `N` bytes, which are pushed into the RX FIFO; each byte dropped because the RX FIFO is full sets `RX_OVERFLOW`.
5. Firmware reads bytes from `RDATA` until `STATUS.RXEMPTY`.

## 8.3 Repeated START

If a new `FDATA` with `START` arrives while a segment is still open (no intervening `STOP`), the open segment is executed first (repeated-START semantics) and a new segment is opened.

## 8.4 Target write / read (back door)

An external controller is emulated with `target_write()` / `target_read()`. A match against either `{ADDRESS0, MASK0}` or `{ADDRESS1, MASK1}` (`addr & mask == address & mask`, `mask ≠ 0`) ACKs the transaction; otherwise it NACKs and increments the saturating `TARGET_NACK_COUNT`. A matched write pushes `start`, one `Data` entry per byte, and (if requested) `stop` into the ACQ FIFO and latches `TARGET_EVENTS.START_DETECT` / `STOP_DETECT`. A matched read pushes a `start` entry and drains up to `nbytes` from the Target TX FIFO.

## 8.5 FIFO thresholds and levels

`HOST_FIFO_CONFIG` / `TARGET_FIFO_CONFIG` program the FMT/RX and TX/ACQ thresholds that drive the threshold interrupts (§9). `HOST_FIFO_STATUS` / `TARGET_FIFO_STATUS` report the live occupancy of each FIFO.

# 9. Interrupts

`INTR_STATE` aggregates 20 sources. The software-visible value is `level_bits | latched_bits | forced_bits`, where:

- **Level bits** are recomputed continuously from state and read-only in `INTR_STATE`.
- **Latched (event) bits** are set by hardware conditions and cleared by writing 1 (W1C).
- **Forced bits** come from `INTR_TEST` (level sources are held while the test bit is set; event sources are latched once).

```
irq_o = (INTR_STATE & INTR_ENABLE) != 0.
```

| Bit | Source                   | Kind                                          |
|-----|--------------------------|-----------------------------------------------|
| 0   | FMT_THRESHOLD            | level (FMT level < FMT_THRESH)                |
| 1   | RX_THRESHOLD             | level (RX level > RX_THRESH)                  |
| 2   | ACQ_THRESHOLD            | level (ACQ level > ACQ_THRESH)                |
| 3   | RX_OVERFLOW              | latched                                       |
| 4   | CONTROLLER_HALT          | level (any CONTROLLER_EVENTS bit set)         |
| 5   | SCL_INTERFERENCE         | latched (not driven by the model)             |
| 6   | SDA_INTERFERENCE         | latched (not driven)                          |
| 7   | STRETCH_TIMEOUT          | latched (not driven)                          |
| 8   | SDA_UNSTABLE             | latched (not driven)                          |
| 9   | CMD_COMPLETE             | latched                                       |
| 10  | TX_STRETCH               | level ( TARGET_EVENTS stretch bits)           |
| 11  | TX_THRESHOLD             | level (TX level < TX_THRESH)                  |
| 12  | ACQ_STRETCH              | level (not modelled → 0)                      |
| 13  | UNEXP_STOP               | latched (not driven)                          |
| 14  | HOST_TIMEOUT             | latched (not driven)                          |
| 15  | SMBALERT                 | latched (not driven; reachable via INTR_TEST) |
| 16  | CONTROLLER_TX_FIFO_ERROR | latched (FMT overflow)                        |
| 17  | CONTROLLER_RX_FIFO_ERROR | latched                                       |
| 18  | TARGET_TX_FIFO_ERROR     | latched (TX overflow)                         |
| 19  | TARGET_RX_FIFO_ERROR     | latched (ACQ overflow)                        |

## 10. Reset behaviour

On assertion of `rst_n_i = 0`:

- All RW/RO storage registers return to their reset value (0).
- Latched interrupts, forced interrupts, `CONTROLLER_EVENTS`, `TARGET_EVENTS`, `TARGET_ACK_CTRL`, and `TARGET_NACK_COUNT` are cleared.
- The controller `halted` flag is cleared.
- All four FIFOs (FMT, RX, TX, ACQ) are emptied.
- Any pending FMT-drain event is cancelled.
- `irq_o` is driven to 0.

## 11. Bus interface (TLM-2.0 / AXI4-Lite)

---

### 11.1 Transport function

The model exports a single `tlm_utils::simple_target_socket<i2c_controller>` named `reg_socket`.  
`b_transport`:

1. Validates the payload: command `TLM_READ_COMMAND` / `TLM_WRITE_COMMAND`; data length exactly 4; address 4-byte aligned and within the window.
2. Adds `access_delay_ns` to the `sc_time&` delay.
3. Dispatches to `reg_read()` / `reg_write()`, performing all register side effects.
4. Returns `TLM_OK_RESPONSE` on success.

### 11.2 Error responses

| Condition                                                     | Response code                           |
|---------------------------------------------------------------|-----------------------------------------|
| Unsupported command (not READ/WRITE)                          | <code>TLM_COMMAND_ERROR_RESPONSE</code> |
| Data length $\neq$ 4                                          | <code>TLM_BURST_ERROR_RESPONSE</code>   |
| Unaligned address, or address $\geq$ <code>WINDOW_SIZE</code> | <code>TLM_ADDRESS_ERROR_RESPONSE</code> |
| Aligned in-window offset with no register decode              | <code>TLM_ADDRESS_ERROR_RESPONSE</code> |

### 11.3 Debug transport

`transport_dbg` performs a no-delay, **side-effect-free** access: reads use `dbg_reg()` (does not pop the RX/ACQ FIFOs or clear `TARGET_NACK_COUNT`); writes reuse the normal decode. Malformed accesses (bad width/alignment/window, or a write to an undecoded offset) return 0.

### 11.4 DMI

DMI is not granted: `RDATA` and `ACQDATA` reads pop FIFOs and `TARGET_NACK_COUNT` is read-clear, so direct memory access would bypass these effects.

### 11.5 AXI sideband

An optional `smc::smc_axi_extension` on the payload is inspected but not enforced by this model; the fabric `axi_filter` performs access control upstream.

---

## 12. Programming model

---

### 12.1 Controller write (poll)

```

write(BASE + 0x10, 0x1);           // CTRL.ENABLEHOST
write(BASE + 0x1C, (addr<<1) | (1<<8)); // FDATA: START, address (write)
write(BASE + 0x1C, 0xAA);          // FDATA: data
write(BASE + 0x1C, 0xBB | (1<<9)); // FDATA: data + STOP
while (!(read(BASE + 0x00) & (1<<9))) // wait CMD_COMPLETE in INTR_STATE
    ;
write(BASE + 0x00, (1<<9));         // W1C CMD_COMPLETE

```

### 12.2 Controller read (poll)

```

write(BASE + 0x10, 0x1);           // CTRL.ENABLEHOST
write(BASE + 0x1C, (addr<<1) | 1 | (1<<8)); // FDATA: START, address (read)
write(BASE + 0x1C, n | (1<<10) | (1<<9)); // FDATA: READB n bytes + STOP
while (!(read(BASE + 0x14) & (1<<5))) // while !STATUS.RXEMPTY
    buf[i++] = read(BASE + 0x18);      // RDATA

```

### 12.3 Target setup

```

write(BASE + 0x54, addr0 | (0x7F<<7)); // TARGET_ID: ADDRESS0, MASK0=0x7F
write(BASE + 0x10, 0x2);               // CTRL.ENABLETARGET
// fill TX FIFO for reads:
write(BASE + 0x5C, 0x77);              // TXDATA
// on a controller access, drain ACQ:
while (!(read(BASE + 0x14) & (1<<9))) // while !STATUS.ACQEMPTY
    entry = read(BASE + 0x58);          // ACQDATA {byte, signal}

```

---

## 13. Compliance matrix

| Feature                                                               | RTL | SystemC model       | Notes                                                          |
|-----------------------------------------------------------------------|-----|---------------------|----------------------------------------------------------------|
| Full register file ( <code>i2c.rdl</code> , <code>0x00..0x80</code> ) | ✓   | ✓                   | Bit-compatible with the RDL layout                             |
| Controller Mode FMT/RX FIFO datapath                                  | ✓   | ✓                   | Segment reassembly + bus-model execution                       |
| START / repeated-START / STOP handling                                | ✓   | ✓                   | Repeated START flushes the open segment                        |
| READB read length (0 ⇒ 256)                                           | ✓   | ✓                   |                                                                |
| NAKOK suppresses halt                                                 | ✓   | ✓                   |                                                                |
| NACK → <code>CONTROLLER_EVENTS</code> + halt/resume                   | ✓   | ✓                   | Cleared via W1C resumes draining                               |
| Target Mode ACQ/TX FIFOs, address match/mask                          | ✓   | ✓                   | Two address+mask pairs                                         |
| Saturating read-clear <code>TARGET_NACK_COUNT</code>                  | ✓   | ✓                   |                                                                |
| FIFO thresholds + level/latched interrupts                            | ✓   | ✓                   | 20-source aggregation                                          |
| FIFO overflow error interrupts                                        | ✓   | ✓                   | FMT / RX / TX / ACQ                                            |
| <code>INTR_TEST</code> force paths                                    | ✓   | ✓                   | Level held; event latched                                      |
| <code>transport_dbg</code> no-side-effect read                        | n/a | ✓                   | Model-only debug path                                          |
| Bit-level SCL/SDA waveform + arbitration                              | ✓   | <b>NOT modelled</b> | Functional/LT abstraction                                      |
| Clock stretching / analog <code>TIMINGx</code> / timeouts             | ✓   | <b>storage only</b> | Firmware-visible, no timing effect                             |
| SMBus alert/suspend signalling                                        | ✓   | <b>storage only</b> | <code>SMBUS_CTRL</code> / <code>SMBUS_STATUS</code> abstracted |
| Multi-controller / monitor bus observation                            | ✓   | <b>NOT modelled</b> | Enable bit is storage                                          |

## 14. Revision history

| Revision | Date       | Author   | Notes                                                                                 |
|----------|------------|----------|---------------------------------------------------------------------------------------|
| 0.1      | 2026-07-14 | SMC team | Initial draft; tracks the <code>i2c</code> core and <code>i2c.rdl</code> register map |

## 15. Glossary

| Term                   | Definition                                                                                                  |
|------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>FMT FIFO</b>        | Controller TX format queue written through <code>FDATA</code> .                                             |
| <b>RX FIFO</b>         | Controller receive queue popped through <code>RDATA</code> .                                                |
| <b>TX FIFO</b>         | Target transmit queue filled through <code>TXDATA</code> .                                                  |
| <b>ACQ FIFO</b>        | Target receive/acquisition queue read through <code>ACQDATA</code> .                                        |
| <b>Segment</b>         | One addressed transaction phase between START/repeated-START and the next START/STOP.                       |
| <b>Bus model</b>       | Test-bench callback emulating the remote target(s) for a Controller-Mode segment.                           |
| <b>W1C</b>             | Write-1-to-clear: writing a 1 clears the corresponding latched bit.                                         |
| <b>Controller halt</b> | State entered on an unexpected NACK; blocks FMT draining until events are cleared.                          |
| <b>LT</b>              | Loosely-Timed. TLM-2.0 modelling style using <code>b_transport</code> with annotated delay.                 |
| <b>QK</b>              | Quantum Keeper ( <code>t1m_utils::t1m_quantumkeeper</code> ); manages temporal decoupling in the initiator. |